

Стать IT-шником может каждый



Индексация, начало работы с матрицами



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Базовые операции с массивами
- Другие арифметические операции над массивами и `inf/nan`
- Сравнения, различные встроенные функции



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Объясните, что происходит в этом фрагменте кода:

```
a = np.array([[0.0, 1.0], [-1.0, 0.0]])  
b = np.array([[0.0, 1.0], [2.0, 3.0]])  
print(a * b)  
# [[ 0.  1.]  
#  [-2.  0.]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Сортировка, изменение массивов

Индексация для доступа к данным

Двумерные массивы и изменение размерности

Умножение матриц



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



Стать IT-шником может каждый



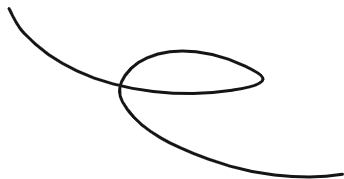
Сортировка, изменение массивов





sort —

это функция, которая возвращает отсортированную копию, метод sort сортирует на месте.



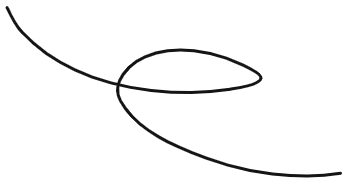
Функция sort

```
b = np.array([6, 4, 10])  
print(np.sort(b)) # [ 4  6 10]  
print(b) # [ 6  4 10]  
b.sort()  
print(b) # [ 4  6 10]
```



delete, insert и append —

это функции, которые не меняют массив на месте, а возвращают новый массив, в котором удалены, вставлены в середину или добавлены в конец какие-то элементы.



Функции delete, insert и append

Функция delete

Удаляет по индексу, а не по значению

Функция insert

Принимает индекс, начиная с которого нужно вставить элемент/элементы и собственно элементы

Функция append

Возвращает новый массив, а исходный массив остается без изменений

Функции delete, insert и append

```
a = np.delete(a, 5)
print(a)
# [1 3 2 4 6]
a = np.insert(a, 2, [0, 0])
print(a)
# [1 3 0 0 2 4 6]
a = np.append(a, [1, 2, 3])
print(a)
# [1 3 0 0 2 4 6 1 2 3]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



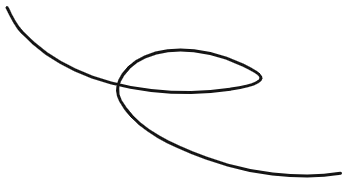
Индексация для доступа к данным





Индексация в NumPy —

это способ доступа к элементам массива. В многомерных массивах для адресации используются индексы для каждого измерения. Если мы хотим получить значение по индексам в многомерном массиве, нужно указывать индексы через запятую внутри квадратных скобок.



Индексация для доступа к данным

```
one_d_array = np.array([1, 2, 3])
print(one_d_array[0]) # 1
two_d_array = np.array([[1, 2, 3], [4, 5, 6]])
print(two_d_array[1]) # Выведет вторую строку [4 5 6]
print(two_d_array[1, 2]) # Выведет 6, элемент во второй строке и третьем столбце
print(two_d_array[1][2]) # Тоже выведет 6, но обычно принято обращаться через
запятую в рамках одних скобок
```

Стать IT-шником может каждый



Индексация для доступа к данным

Массивы являются изменяемыми объектами

Индексация для доступа к данным

```
int_array = np.array([1, 2, 3])  
int_array[2] = 5  
print(int_array)  
# [1 2 5]
```

Срезы

```
print(one_d_array[0:2]) # Выведет [1 2]
print(two_d_array[0:2]) # [[1 2 3], [4 5 6]] – выведет с 0 по 2 строку не вкл
print(two_d_array[0:2, 0:2]) # [[1 2], [4 5]] – выведет с 0 по 2 строку не вкл и с 0 по 2
столбец не вкл
```

Размерности

```
print(two_d_array[0, :]) # [1 2 3] – первая строка  
print(two_d_array[:, 1]) # [2 5] – второй столбец
```

Размерности

```
two_d_array[0, :]
```

```
two_d_array[0]
```

Размерности

```
x = np.array([1.1, 2.2, 3.3, 4.4, 5.5])  
s = x[1:3:1]  
print(s)  
# [2.2 3.3]  
s[0] = 101  
print(s)  
# [101.  3.3]
```

Экспресс-опрос



Как вы думаете, что произойдет с исходным массивом x ?

Размерности

```
print(x)  
# array([ 1.1, 101.,  3.3,  4.4,  5.5])
```



Срезы

- В `numpy` срез — это не новый массив, содержащий те же элементы, что старый, а `view` (вид), то есть интерфейс к старому массиву.
- Срез `s` помнит, что «его» элемент с индексом 0 — это на самом деле элемент с индексом 1 от исходного массива `x`, а его элемент с индексом 1 — это на самом деле элемент с индексом 2 от исходного массива, а других элементов у него нет.
- Преимущество: если есть огромный массив данных, то не придётся тратить ресурсы на то, чтобы его скопировать, если нам нужно сделать срез.
- Недостаток: ухудшается читаемость программы.

Создание копии массива

```
y = x.copy()
```

Работа с обычными списками

```
x = [1.1, 2.2, 3.3, 4.4, 5.5]
s = x[1:3:1]
print(s)
# [2.2, 3.3]
s[0] = 101
print(s)
# [101, 3.3]
print(x)
# [1.1, 2.2, 3.3, 4.4, 5.5]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Что выведет следующий код:

```
cube = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  
print(cube[1, 0, 1])  
print(cube[:, 1, 0])  
print(cube[:, 1])
```

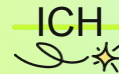
Двумерные массивы и изменение размерности



Двумерные массивы и изменение размерности

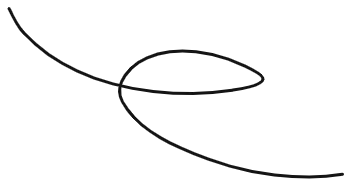
```
b = np.linspace(0, 3, 4)
print(b)
# [0. 1. 2. 3.]
print(b.shape)
# (4,)
b.shape = 2, 2
print(b)
# [[0. 1.]
#   [2. 3.]]
```

Полезно знать



reshape —

этот метод используется для изменения формы (размерности) массива без изменения его данных (элементов).



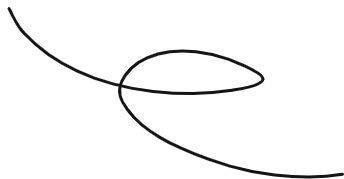
reshape

```
print(np.linspace(0, 3, 4).reshape((2, 2)))  
# [[0. 1.]  
#  [2. 3.]]
```



ravel и flatten —

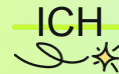
это функции, с помощью которых можно растянуть многомерный массив в одномерный.



Функции ravel и flatten

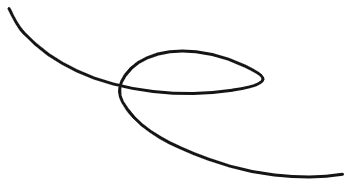
```
print(b.ravel())  
# [0. 1. 2. 3.]  
print(b.flatten())  
# [0. 1. 2. 3.]
```

Полезно знать

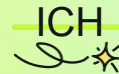


Матрица —

это **двумерный массив**, который обладает великим множеством свойств и приложений.

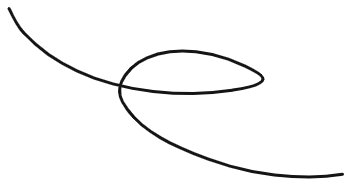


Полезно знать



Квадратная матрица —

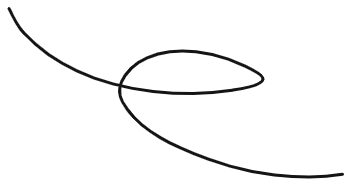
это матрица, у которой количество строк и столбцов одинаково.





Транспонированная матрица —

это матрица, в которой строки и столбцы поменяны местами.



Матрицы

Матрица

1	2
3	4
5	6

Транспонированная матрица

1	3	5
2	4	6

Транспонирование матрицы в numpy

```
print(b)
print(b.T)
# [[0. 1.]
#  [2. 3.]]
# [[0. 2.]
#  [1. 3.]]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Умножение матриц



Умножение вектор-строки на вектор-столбец

```
a = np.array([1, 2, 3])  
b = np.array([[11], [12], [13]])  
print(a * b)  
# [[11 22 33]  
# [12 24 36]  
# [13 26 39]]
```

Операция матричное умножение

```
a = np.array([1, 2, 3])  
b = np.array([[11], [12], [13]])  
print(a @ b)  
# [74]
```

Операция матричное умножение

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 11 \\ 12 \\ 13 \end{bmatrix} = \begin{bmatrix} 1 * 11 + 2 * 12 + 3 * 13 \end{bmatrix}$$

Операция матричное умножение

```
a = np.array([[1, 2, 3], [4, 0, 5]])  
b = np.array([[11], [12], [13]])  
print(a @ b)
```

```
# [[ 74]  
# [109]]
```

Операция матричное умножение

$$\begin{bmatrix} \boxed{1} & \boxed{2} & \boxed{3} \\ \boxed{4} & \boxed{0} & \boxed{5} \end{bmatrix} \begin{bmatrix} \textcircled{11} \\ \textcircled{12} \\ \textcircled{13} \end{bmatrix} = \begin{bmatrix} \boxed{1} * \textcircled{11} + \boxed{2} * \textcircled{12} + \boxed{3} * \textcircled{13} \\ \boxed{4} * \textcircled{11} + \boxed{0} * \textcircled{12} + \boxed{5} * \textcircled{13} \end{bmatrix}$$

Операция матричное умножение

```
a = np.array([[1, 2, 3], [4, 0, 5]])  
b = np.array([[11, 100], [12, 200], [13, 300]])  
print(a @ b)
```

```
# [[ 74 1400]  
# [ 109 1900]]
```

Операция матричное умножение

The diagram illustrates matrix multiplication. On the left, a 2x3 matrix is shown with a green top row [1, 2, 3] and a blue bottom row [4, 0, 5]. This is multiplied by a 3x2 matrix with a pink left column [11, 12, 13] and a yellow right column [100, 200, 300]. The result is a 2x2 matrix with a green top row [74, 1400] and a blue bottom row [109, 1900]. The green and blue colors in the result matrix correspond to the colors of the rows in the first matrix, indicating row-major multiplication.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 5 \end{bmatrix} \begin{bmatrix} 11 & 100 \\ 12 & 200 \\ 13 & 300 \end{bmatrix} = \begin{bmatrix} 74 & 1400 \\ 109 & 1900 \end{bmatrix}$$

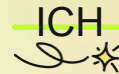
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Напишите код:

Умножить матрицу b на матрицу a .

$$\begin{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \\ \begin{bmatrix} 4 & 0 & 5 \end{bmatrix} \end{bmatrix} \begin{bmatrix} \begin{bmatrix} 11 \\ 12 \\ 13 \end{bmatrix} \\ \begin{bmatrix} 100 \\ 200 \\ 300 \end{bmatrix} \end{bmatrix}$$

a

b



Напишите код:

Умножить матрицу a на матрицу b и наоборот .

0	1
-1	0

a

0	1
2	3

b

Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание 1

Что будет выведено в следующих случаях и почему:

```
cube = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  
  
print(cube[:, 0])  
  
print(cube[1, :, 0])  
  
s = cube[:, 1]  
  
s *= 10  
  
print(s)  
  
print(cube)
```

Практическое задание 2

Умножьте матрицу a на b из отрывка кода ниже. Сначала самостоятельно, потом напишите код на Python. Прodelайте аналогичные шаги для умножения матрицы b на матрицу a .

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Полезные ссылки

- [Умножение матриц — Википедия](#)
- [Индексация и срезы в NumPy](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

